Spoken Language Identification Using Residual CNNs and Log-Mel Spectrograms

Biniyam Zergaw
Department of Computer and Information Science
Fordham University
New York, NY, USA
bz24@fordham.edu

***Abstract*—Spoken language identification is the task of determining which language is being spoken from a short audio recording. This paper presents a neural network system that classifies audio into one of nine languages: Amharic, Arabic, Chinese, English, French, Hindi, Italian, Russian, and Spanish. We use 128-bin log-Mel spectrograms as input features. Two architectures are evaluated: a residual convolutional neural network (CNN) as the primary model, and a bidirectional long short-term memory (LSTM) network as a sequence-based baseline. Training data is drawn from Mozilla Common Voice and Google FLEURS, with up to 15,000 clips per language. On the held-out test set, the residual CNN achieves 89.3% accuracy, outperforming the LSTM baseline at 80.5%. Analysis of confusion matrices and PCA embeddings indicates that the CNN learns more separable language representations from the log-Mel spectrograms, though both models show reduced accuracy on live microphone recordings due to domain mismatch.**

***Index Terms*—spoken language identification, audio classification, convolutional neural networks, LSTM, log-Mel spectrograms, PyTorch**

I. Introduction

Spoken language identification (LID) is the task of automatically determining the language being spoken in an audio recording. It is a foundational component in language-specific automatic speech recognition (ASR), call routing in customer service systems, transcription systems, and other applications that must first identify a language before applying language-specific speech recognition. Unlike text-based language detection, spoken LID must operate on raw acoustic signals, requiring models to capture phonetic, prosodic, and rhythmic patterns that differentiate languages without access to lexical content.

This paper evaluates two neural network architectures for nine-language spoken LID. The primary model is a five-block residual CNN trained on log-Mel spectrograms, and a bidirectional LSTM is used as a sequence-modeling baseline. Training data combines Mozilla Common Voice and Google FLEURS recordings to improve cross-domain robustness. The central question addressed is whether local time-frequency pattern modeling via convolution outperforms frame-level sequential modeling via recurrence for spoken LID task, and how both approaches generalize beyond the training distribution to live microphone recordings.

II. Related Work

Spoken language identification has been studied for several decades. Early systems relied on acoustic-phonetic and phonotactic approaches, including GMM tokenizers and parallel phone recognition followed by language modeling (PRLM) [3]. I-vectors [4], originally developed for speaker verification, were adapted for LID [5] and became the standard fixed-dimensional utterance embedding through the mid-2010s.

Deep learning methods began to outperform classical approaches as data scaled. Lopez-Moreno et al. [6] showed that a Deep Neural Network (DNN) trained on acoustic features surpassed GMM and i-vector baselines across 26 languages, and x-vectors [7], TDNN-based embeddings originally designed for speaker recognition, later further advanced LID performance on benchmarks.

CNNs applied to log-mel spectrograms have since become a standard approach for audio classification. Lozano-Diez et al. [8] demonstrated that convolutional feature learning over time-frequency representations can match or exceed hand-crafted feature systems. More recently, large self-supervised models such as wav2vec 2.0 [9], HuBERT, and Whisper have raised performance ceilings by pretraining on large multilingual audio corpora. The present work does not use pretrained encoders; instead, it evaluates a residual CNN and a bidirectional LSTM trained from scratch as a controlled comparison of architectural inductive biases for log-mel spectrogram classification.

III. Problem Formulation

Spoken language identification is formulated as a supervised multi-class classification problem. Let x denote an audio waveform and let $y \in \{1, \ldots, K\}$ denote the corresponding language label, where $K = 9$. Each waveform is resampled to 16 kHz, trimmed or padded to a fixed five-second duration, and transformed into a log-Mel spectrogram before being passed to a neural network classifier. The classifier produces a probability distribution over the K language classes, and the predicted label is taken as:

$$\hat{y} = \arg\max_{k} p(y = k \mid x). \tag{1}$$

TABLE I
DATASET SPLIT SIZES

Split	Number of clips
Training	80,371
Validation	17,222
Test	17,223

The model is trained by minimizing the cross-entropy loss over the training set:

$$\mathcal{L} = -\sum_{k=1}^{K} y_k \log(\hat{p}_k), \tag{2}$$

where y_k is the one-hot encoded true label, and $\hat{p}_k$ is the predicted probability for class k.

IV. METHODOLOGY

A. Dataset

The dataset is organized by language and draws from two publicly available speech corpora. The primary source is Mozilla Common Voice [1], which provides crowd-sourced recordings across a wide range of speakers. The secondary source is Google FLEURS [2], a multilingual benchmark dataset that introduces a distinct recording domain and helps evaluate cross-dataset robustness. For Amharic, an auxiliary speech corpus was used to supplement the Mozilla data, which was substantially smaller for that language than for the others.

Each language was capped at 10,000 clips from Mozilla Common Voice, 5,000 clips from FLEURS, and 15,000 total clips before filtering and splitting. The dataset loader identifies FLEURS clips by directory structure, allowing both sources to be included in training while remaining distinguishable for analysis. After filtering for audio quality and applying source availability constraints, the final dataset comprises 114,816 clips divided into training, validation, and test splits as shown in Table I.

B. Feature Extraction and Preprocessing

Log-Mel spectrograms are used as the primary model input rather than raw waveforms. Each audio file is loaded using librosa [11], converted to mono, and resampled to 16 kHz. A fixed five-second representation is produced by trimming or zero-padding as needed. A 128-bin Mel-filterbank spectrogram is then computed with an FFT size of 1,024 and a hop length of 512 samples, and the resulting power spectrogram is converted to decibel scale and normalized. Log-Mel spectrograms were chosen because they preserve local time-frequency structure capturing harmonics, formants, and phonetic transitions in a two-dimensional format that is well-suited to convolutional processing. Mel-Frequency Cepstral Coefficients (MFCC) features were implemented as an alternative but were not used in the final experiments.

Several data augmentation strategies were incorporated to improve generalization. During training, SpecAugment [13]-style frequency and time masking is applied to cached spectrograms. A heavier augmentation path that included random gain adjustment, background noise addition, pitch shifting, and time stretching was implemented but not used in the final pipeline due to the substantial increase in per-epoch training time it introduced. Silence trimming via voice activity thresholding and random five-second cropping are also available as preprocessing options.

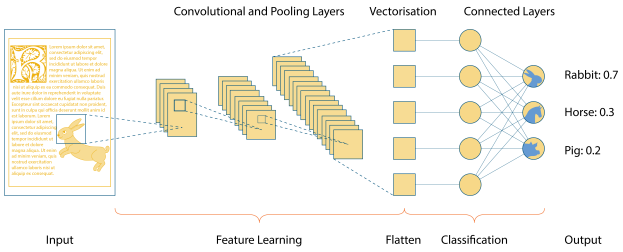

Fig. 1. General convolutional neural network structure [12].

C. Model Architectures

1) Residual CNN: The primary model is a residual convolutional neural network trained on log-Mel spectrograms. The input is a single-channel spectrogram of shape $(1, 128, T)$, where 128 denotes the number of Mel frequency bands and T is the number of time frames corresponding to a five-second clip at 16 kHz. The network consists of five convolutional blocks with progressively increasing channel depths: $1 \to 32$, $32 \to 64$, $64 \to 128$, $128 \to 256$, and $256 \to 512$. Each block applies two 3×3 convolutions, batch normalization, and ReLU activation, with a residual skip connection that adds the block input to its output. Max pooling reduces spatial resolution in the earlier blocks, and adaptive average pooling compresses the final feature map into a compact 512-dimensional representation. A three-layer fully connected classifier then maps this 512-dimensional representation to the nine output classes through layers of size 256 and 128, with dropout applied between layers for regularization and the residual CNN contains approximately 4.08M trainable parameters.

This architecture is well-suited to spectrogram classification because convolutional filters can detect local time-frequency patterns such as harmonic structures and phonetic transitions and compose them hierarchically into language-level representations. The residual connections mitigate vanishing gradient issues during training and allow the network to learn identity mappings when additional depth provides no benefit. The general structure of a convolutional neural network is illustrated in Fig. 1.

2) Bidirectional LSTM Baseline: The baseline model is a bidirectional long short-term memory (LSTM) network. The log-Mel spectrogram is interpreted as a temporal sequence, where each time frame is represented by its 128 Mel-band coefficients. The network comprises two bidirectional LSTM layers with a hidden size of 128, producing a 256-dimensional representation at each time step (128 per direction). The final

TABLE II
MAIN TRAINING CONFIGURATION

Parameter	Value
Sample rate	16 kHz
Clip duration	5 seconds
Mel bands	128
Batch size	64
Learning rate	3×10^{-4}
Weight decay	1×10^{-4}
Maximum epochs	30
Early stopping patience	8
Training workers	4

hidden state is passed to a fully connected classifier that maps to the nine output classes.

The LSTM was included as a baseline because recurrent architectures are designed for sequential data and provide a natural alternative to the spatial processing of CNNs. However, recurrent operations are inherently less parallelizable than convolutions, resulting in slower training for a comparable number of parameters. More importantly, the LSTM processes the spectrogram strictly as a sequence of frames and does not explicitly model relationships across frequency bands within a single frame, which limits its ability to capture the local time-frequency structure that distinguishes languages acoustically. The bidirectional LSTM contains approximately 693K trainable parameters. The implications of this architectural difference are examined in the results and discussion sections.

D. Training Setup

Both models were trained using PyTorch [10] with the Adam optimizer [15] and cross-entropy loss. A learning rate of 3×10^{-4} and weight decay of 1×10^{-4} were applied throughout training. The full set of hyperparameters is summarized in Table II.

To make training computationally practical, log-Mel spectrograms were precomputed and cached to disk prior to training. This eliminates repeated audio decoding and feature extraction across epochs, significantly reducing per-epoch wall-clock time. During training, SpecAugment [13]-style frequency and time masking was applied on-the-fly to the cached spectrograms to provide regularization through input variation. A heavier live augmentation path including pitch shifting, time stretching, random gain, and background noise addition was available but excluded from the final pipeline due to the substantial per-epoch overhead it introduced.

Early stopping with a patience of 8 epochs was used to halt training when validation loss ceased to improve, preventing overfitting and reducing unnecessary computation. Both models were trained for a maximum of 30 epochs with a batch size of 64 and 4 parallel data-loading workers. The CNN and LSTM were trained independently under identical hyperparameter settings to ensure a fair comparison.

V. EVALUATION

Model performance is assessed on the held-out test split of 17,223 clips, which was not used at any point during training or hyperparameter selection. The primary metric is overall classification accuracy. Per-class precision, recall, and F1-score are also reported to capture language-level variation in model behavior that overall accuracy alone would obscure.

Confusion matrices are used to identify systematic misclassification patterns between languages. This is particularly informative in a multilingual setting, where acoustically or prosodically similar language pairs such as the Romance languages French, Italian, and Spanish are expected to produce higher confusion rates than typologically distant pairs.

Principal Component Analysis (PCA) is applied to the 512-dimensional embeddings extracted from the penultimate layer to visualize the geometry of the learned representation space. Well-separated clusters in the PCA projection indicate that the model has learned discriminative language representations, while overlapping regions identify language pairs that remain difficult to separate.

In addition to the held-out test set, a secondary evaluation is conducted on a set of generated sample predictions drawn from `samples/predict_test/`, which tests generalization to audio from sources outside the training distribution. A qualitative evaluation on live microphone recordings is also performed to assess real-world usability, though this is not included in the quantitative results due to the absence of ground-truth labels for user-recorded audio.

TABLE III
INITIAL SMALL-DATA TEST PERFORMANCE

Language	Precision	Recall	F1
Italian	0.58	0.84	0.69
Russian	0.79	0.48	0.59
Amharic	0.92	0.81	0.86
Hindi	0.78	0.80	0.79
Overall	≈70–72% accuracy		

VI. RESULTS

A. Initial Small-Data Sanity Test

Before training the final nine-language system, a small-data experiment was conducted to validate the architecture and training pipeline on a reduced language set. This run used four languages and a substantially smaller clip count. The goal was not to produce a deployment-ready model, but to confirm that the network could learn language-discriminative representations from log-Mel spectrograms at all.

The model learned the training distribution but showed clear signs of overfitting and weak generalization. On the held-out test set, it reached approximately 70–72% accuracy across the four languages. Per-class results are shown in Table III. Amharic was the strongest class. Russian had low recall, indicating that many Russian clips were misclassified as another language. Italian had high recall but low precision, meaning the model frequently predicted Italian for clips that were not Italian which is a classic symptom of a biased decision boundary.

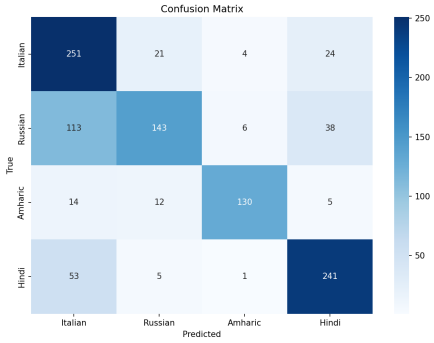

Fig. 2. Confusion matrix for the initial CNN model.

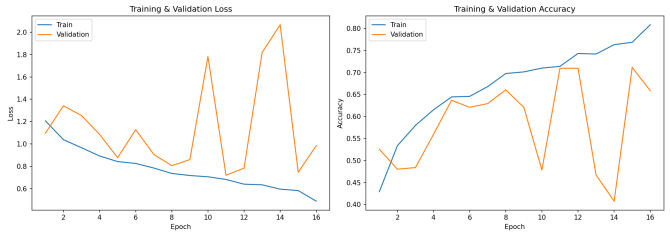

Fig. 3. Training and validation curves for the initial CNN model, showing signs of overfitting.

On the generated sample test, only 24 of 45 clips were classified correctly (53.3%). This gap between split accuracy and sample accuracy indicated that the model was not merely underfitting but rather it was learning dataset-specific patterns rather than language-general acoustic features. These findings motivated the subsequent changes: expanding to nine languages, increasing clip counts, incorporating FLEURS data, refining preprocessing, and introducing the LSTM as a comparative baseline. The initial confusion matrix and learning curves are shown in Figures 2 and 3.

B. CNN Results

The residual CNN achieved 89.3% overall accuracy on the held-out test set of 17,223 clips. Per-language precision, recall, and F1-scores are reported in Table IV. Amharic, Chinese, and Hindi were the strongest classes, each achieving F1-scores of 0.98, 0.92, and 0.92 respectively. English was the weakest, with a recall of 0.80, suggesting that a notable fraction of English clips were assigned to neighboring language classes. The CNN confusion matrix and learning curves are shown in Figures 4 and 5.

C. LSTM Results

The bidirectional LSTM achieved 80.5% overall accuracy on the held-out test set. Per-language results are shown in Table V. Amharic remained the strongest class with an F1-score of 0.97, consistent with the CNN results. English again showed the lowest F1-score at 0.74, with precision dropping to 0.67 the lowest of any class across both models. French showed an asymmetric pattern of high precision (0.89) but low recall (0.73), indicating that the LSTM was selective

TABLE IV
CNN HELD-OUT TEST PERFORMANCE

Language	Precision	Recall	F1
Amharic	0.98	0.98	0.98
Arabic	0.91	0.88	0.89
Chinese	0.91	0.94	0.92
English	0.87	0.80	0.83
French	0.86	0.90	0.88
Hindi	0.91	0.92	0.92
Italian	0.89	0.86	0.87
Russian	0.86	0.90	0.88
Spanish	0.85	0.85	0.85
Overall	89.3% accuracy		

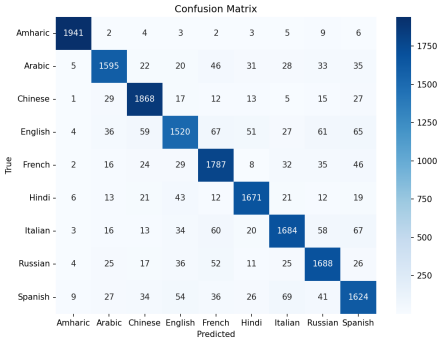

Fig. 4. Confusion matrix for the residual CNN model.

about predicting French but missed a substantial portion of true French clips. The LSTM confusion matrix and learning curves are shown in Figures 6 and 7.

The training curves in Figure 7 show an unusual pattern in which validation accuracy consistently exceeds training accuracy throughout training. This is an artifact of dropout regularization: during training, dropout is active and randomly zeros 30% of activations on every forward pass, artificially suppressing training metrics. During validation, dropout is disabled and the model operates at full capacity, producing higher measured accuracy. This behavior is expected and indicates that regularization is functioning as intended rather than that the model generalizes better than it learns.

D. Model Comparison

Table VI summarizes the overall accuracy of both models on the held-out test set and the generated sample prediction

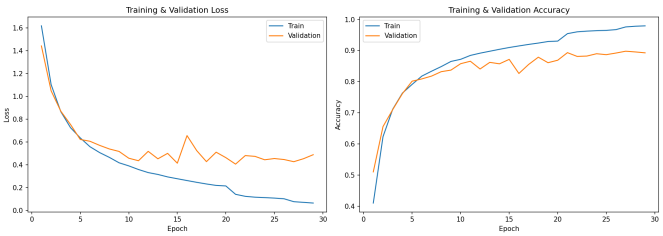

Fig. 5. Training and validation curves for the residual CNN model.

TABLE V
LSTM HELD-OUT TEST PERFORMANCE

Language	Precision	Recall	F1
Amharic	0.97	0.96	0.97
Arabic	0.76	0.79	0.77
Chinese	0.80	0.88	0.84
English	0.67	0.83	0.74
French	0.89	0.73	0.80
Hindi	0.84	0.83	0.84
Italian	0.74	0.77	0.75
Russian	0.82	0.75	0.78
Spanish	0.79	0.71	0.75
Overall	80.5% accuracy		

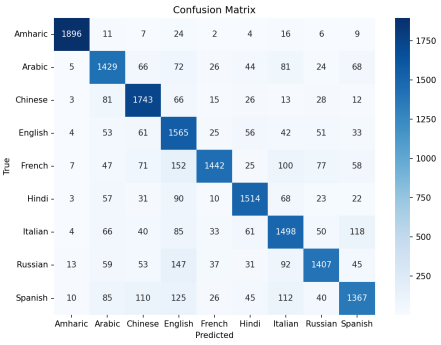

Fig. 6. Confusion matrix for the bidirectional LSTM model.

set. The residual CNN outperforms the bidirectional LSTM on both evaluation conditions, with an 8.8 percentage point advantage on the held-out test set and a 13.3 percentage point advantage on the recorded sample set. The larger gap on the sample set suggests that the CNN not only achieves higher in-distribution accuracy but also generalizes more robustly to audio from outside the training distribution.

VII. Interpretation and Discussion

The results demonstrate that the residual CNN is the stronger model for spoken language identification from log-Mel spectrograms. It achieves 89.3% held-out accuracy across nine typologically diverse languages, maintaining a consistent advantage over the bidirectional LSTM baseline at every level of evaluation, which includes held-out test set, generated samples, and qualitative microphone recordings.

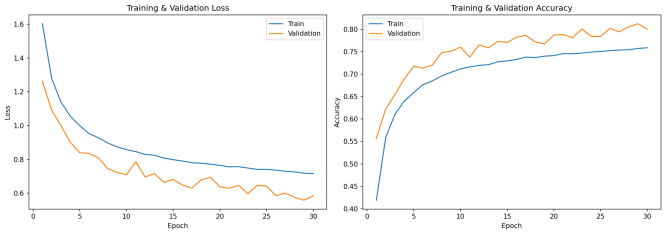

Fig. 7. Training and validation curves for the bidirectional LSTM model.

TABLE VI
CNN AND LSTM COMPARISON

Model	Held-out accuracy	Sample accuracy
Residual CNN	89.3%	84.4%
Bidirectional LSTM	80.5%	71.1%

A. Per-Language Performance Patterns

The languages best classified by the CNN which are Amharic (F1: 0.98), Chinese (F1: 0.92), and Hindi (F1: 0.92) share a common property: they are acoustically and phonologically distinct from the other eight languages in the dataset. Amharic uses a unique set of ejective consonants and a phonological system unrelated to any Indo-European language, making it highly separable in acoustic feature space. Chinese is tonal, imposing distinctive pitch contour patterns on every syllable that do not appear in the other languages. Hindi, while Indo-European, has a consonant inventory including retroflex stops and aspirated consonants that produces spectrographic signatures not shared by the European languages in the dataset.

English was the weakest CNN class (F1: 0.83, recall: 0.80), which is noteworthy given that all languages were sampled at the same clip cap from Mozilla Common Voice. One likely explanation is that English, as a globally spoken language, exhibits extreme within-class acoustic variation: accents, speaking rates, and recording conditions vary more widely for English than for most other languages in the dataset. This within-class variation makes English clips harder to cluster tightly in representation space, as confirmed by the broader English distribution visible in the PCA embedding in Figure 9.

The Romance languages French, Italian, and Spanish showed moderate and broadly consistent F1-scores in the CNN (0.88, 0.87, and 0.85 respectively), reflecting the shared phonetic and prosodic properties of this language family. Figure 8 illustrates the lexical proximity of European languages; spoken confusion among them is also driven by shared phonotactics, similar vowel inventories, and comparable rhythmic patterns. The confusion matrix in Figure 4 confirms that the majority of misclassifications among these three languages are cross-Romance rather than cross-family.

B. CNN vs. LSTM: Inductive Bias and Representation

It is worth noting that the residual CNN (4.08M parameters) has approximately six times more capacity than the bidirectional LSTM (693K parameters). The performance gap therefore reflects both an architectural advantage and a difference in model scale; however, the LSTM's structural inability to model relationships across frequency bins within a single frame remains a fundamental limitation that additional parameters alone would not resolve. It reflects a fundamental mismatch between the LSTM's inductive bias and the structure of the input. A log-Mel spectrogram is a two-dimensional array in which both the time axis and the frequency axis carry structured, locally correlated information. Convolutional filters operate over local receptive fields in both dimensions simultaneously, learning features such as formant transitions,

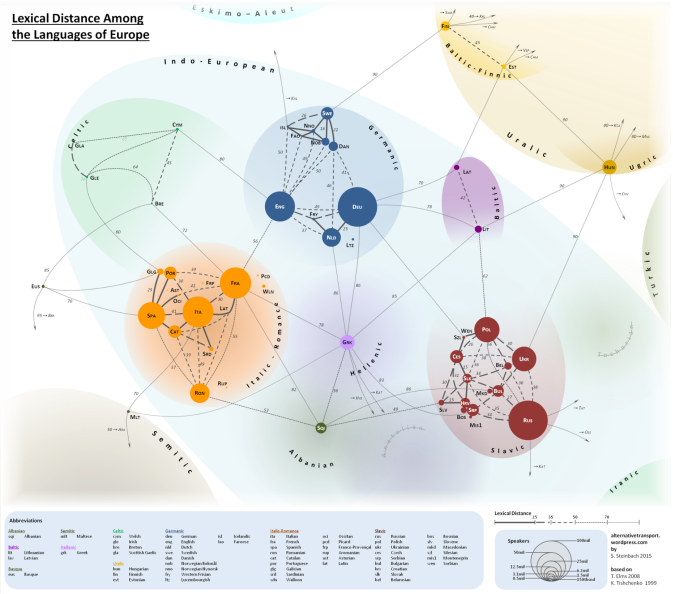

Fig. 8. Lexical distances between European languages [14].

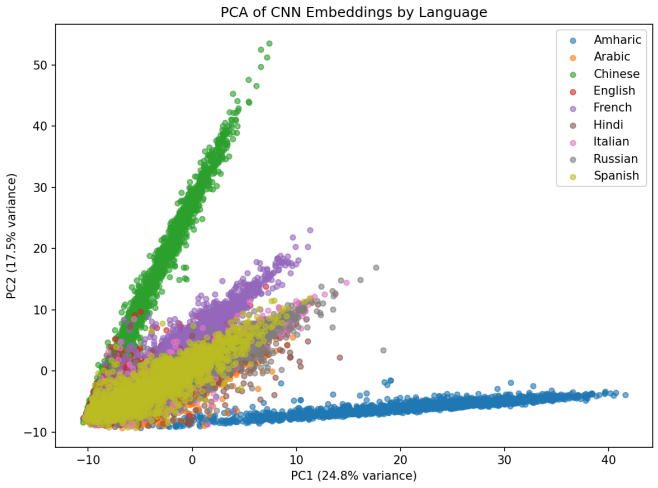

Fig. 9. PCA visualization of CNN embeddings for the nine-language classification system.

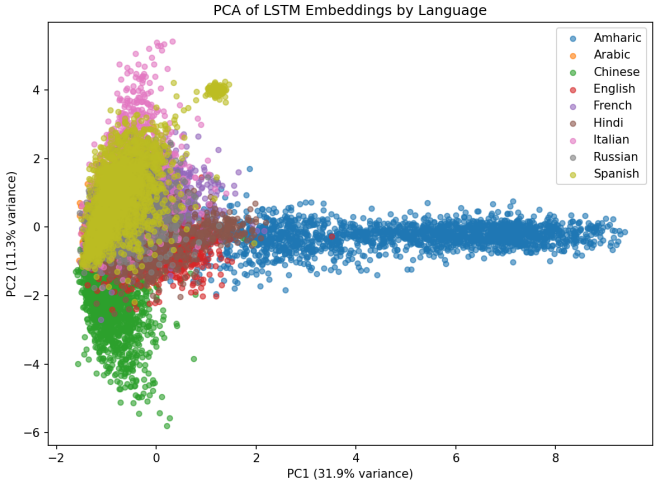

Fig. 10. PCA visualization of LSTM embeddings for the nine-language classification system.

harmonic patterns, and spectral envelopes that are defined by relationships across both time and frequency. The LSTM, by contrast, collapses the frequency dimension into a single vector at each time step and processes those vectors sequentially. This means it cannot directly model within-frame frequency relationships, and must instead rely on temporal context alone to accumulate language-level information.

This architectural difference is visible in the per-language results. The LSTM's weakest class was English (F1: 0.74, precision: 0.67), and its largest precision-recall asymmetry appeared in French (precision: 0.89, recall: 0.73). These patterns suggest that the LSTM is forming less stable decision boundaries than the CNN, likely because it lacks the two-dimensional local feature detectors needed to reliably separate acoustically similar languages. The CNN's residual connections further contribute by allowing the network to preserve low-level spectral features across depth, which would otherwise be lost in a purely sequential model.

C. Representation Space and PCA Analysis

The PCA projection of CNN embeddings in Figure 9 provides complementary geometric evidence for the quantitative results. Two languages stand out as clearly separated in the two-dimensional projection: Amharic forms a distinct linear cluster along the positive PC1 axis, while Chinese occupies a well-separated region along the positive PC2 axis. Both separations are consistent with their high F1-scores in Table IV. The remaining seven languages namely Arabic, English, French, Hindi, Italian, Russian, and Spanish cluster closely near the origin with substantial overlap, reflecting the higher confusion rates observed among them in the confusion matrix in Figure 4.

That PC1 (24.8%) and PC2 (17.5%) together account for only 42.3% of the total variance indicates that language identity is distributed across a high-dimensional subspace, with no single acoustic dimension cleanly separating all nine languages simultaneously.

The LSTM embedding plot in Figure 10 tells a similar but weaker story. Amharic again separates clearly along PC1 (31.9%), and Chinese separates along PC2 (11.3%), but the remaining languages form an even denser and more entangled mass near the origin compared to the CNN. This geometric difference is consistent with the LSTM's lower overall accuracy and suggests that the CNN learns more discriminative internal representations for the acoustically similar European language group, even when both models handle the more distinctive languages comparably.

D. Domain Mismatch and Real-World Generalization

The drop in accuracy from the held-out test set to real microphone recordings is the most practically significant finding of this study. The held-out test split is sampled from

the same dataset distribution as training, so its accuracy reflects in-distribution generalization. Microphone recordings, by contrast, introduce conditions absent from the training data: different microphone hardware, background noise, room acoustics, spontaneous pronunciation, and varying silence ratios before and after speech onset. This domain gap explains why the CNN reaches 89.3% on the test set but makes errors on user-recorded audio that the test-set accuracy would not predict.

This finding has a direct implication for deployment: a model trained exclusively on curated public speech datasets will not automatically transfer to real-world microphone conditions without either target-domain fine-tuning or substantially stronger augmentation that simulates those conditions during training.

VIII. Limitations

The primary limitation is sensitivity to recording domain. Both models were trained on curated public speech datasets and show measurably reduced accuracy on real microphone recordings, a gap confirmed by the qualitative evaluation. A more robust system would require target-domain recordings or stronger augmentation simulating real-world acoustic conditions.

A second limitation is that both models were trained from scratch on a fixed feature representation. Modern speech systems often use pretrained speech encoders such as wav2vec 2.0 [9], HuBERT, WavLM, or Whisper, which have already learned robust speech representations from very large datasets. Fine-tuning any of these encoders for the nine-language classification task would likely yield substantially higher accuracy, particularly on out-of-domain and noisy microphone recordings.

A third limitation concerns the LSTM baseline. While useful for comparison, the bidirectional LSTM used here does not represent the strongest possible sequence model. Attention-based architectures such as the Transformer, or hybrid convolutional-recurrent models, could learn richer temporal representations than a two-layer LSTM and would provide a more competitive upper bound for sequence-based spoken LID.

IX. Future Work

The most impactful near-term improvement would be to replace the from-scratch CNN with a fine-tuned pretrained speech encoder. Models such as wav2vec 2.0, HuBERT, WavLM, or Whisper have already learned general-purpose speech representations from large multilingual corpora, and adapting them to the nine-language classification task would likely close much of the gap between held-out test accuracy and real microphone performance.

Beyond transfer learning, several other directions could improve robustness. Expanding the training dataset with more diverse target-domain recordings would reduce the domain mismatch identified in Section VII. Stronger data augmentation during training, including room impulse response convolution and realistic noise mixing, could simulate these conditions without requiring additional labeled data. Multi-window inference, in which predictions are averaged across several overlapping time windows from the same recording, could also improve accuracy on longer or noisier clips by reducing the influence of any single poorly-conditioned window.

Finally, expanding the language set beyond nine languages would test the scalability of the current architecture and reveal whether the per-language patterns observed here, particularly the advantage of phonologically distinctive languages, generalize to a broader typological sample.

X. Conclusion

This paper presented a spoken language identification system that classifies short audio recordings into one of nine languages using log-Mel spectrograms and deep neural networks. A residual CNN and a bidirectional LSTM were trained and evaluated on a combined dataset of over 114,000 clips drawn from Mozilla Common Voice and Google FLEURS audio files. The residual CNN achieved 89.3% accuracy on the held-out test set and 84.4% accuracy on generated sample predictions, outperforming the LSTM baseline at 80.5% and 71.1% respectively.

The results confirm that convolutional architectures are better suited than the tested recurrent baseline for log-Mel spectrogram classification. The CNN's advantage stems from its ability to detect local time-frequency patterns across both the time and frequency axes simultaneously; a structural match to the two-dimensional nature of the spectrogram input that the LSTM's sequential processing cannot replicate. PCA analysis of the learned embeddings corroborates this finding geometrically: the CNN produces more separated language clusters, particularly for the acoustically similar European languages that the LSTM struggles to discriminate.

The primary open challenge is domain generalization. Both models show reduced accuracy on real microphone recordings, a gap that held-out test accuracy alone does not reveal. Closing this gap through pretrained encoder fine-tuning, target-domain data collection, or stronger acoustic augmentation represents the most promising direction for future work.

References

[1] Mozilla Foundation, "Common Voice Dataset," Mozilla Common Voice, 2024. [Online]. Available: https://commonvoice.mozilla.org/

[2] A. Conneau et al., "FLEURS: Few-Shot Learning Evaluation of Universal Representations of Speech," in *Proc. IEEE Spoken Language Technology Workshop (SLT)*, 2022, pp. 798–805.

[3] M. A. Zissman, "Comparison of four approaches to automatic language identification of telephone speech," *IEEE Transactions on Speech and Audio Processing*, vol. 4, no. 1, pp. 31–44, Jan. 1996.

[4] N. Dehak, P. J. Kenny, R. Dehak, P. Dumouchel, and P. Ouellet, "Front-end factor analysis for speaker verification," *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 19, no. 4, pp. 788–798, May 2011.

[5] D. Martinez, O. Plchot, L. Burget, O. Glembek, and P. Matejka, "Language recognition in iVectors space," in *Proc. Interspeech*, 2011, pp. 861–864.

[6] I. Lopez-Moreno et al., “Automatic language identification using deep neural networks,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2014, pp. 5337–5341.
[7] D. Snyder, D. Garcia-Romero, G. Sell, D. Povey, and S. Khudanpur, “X-vectors: Robust DNN embeddings for speaker recognition,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2018, pp. 5329–5333.
[8] A. Lozano-Diez, R. Zazo, J. Gonzalez-Dominguez, D. T. Toledano, and J. Gonzalez-Rodriguez, “An analysis of the influence of deep neural network (DNN) topology in bottleneck feature based language recognition,” *PLOS ONE*, vol. 12, no. 8, 2017.
[9] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, “wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 12449–12460.
[10] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
[11] B. McFee et al., “librosa: Audio and Music Signal Analysis in Python,” in *Proceedings of the 14th Python in Science Conference*, 2015, pp. 18–25.
[12] Wikimedia Commons, “Convolutional Neural Network,” 2023. [Online]. Available: https://commons.wikimedia.org/wiki/File:Convolutional_Neural_Network.png [Accessed: May 2025].
[13] D. S. Park et al., “SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition,” in *Proc. Interspeech*, 2019, pp. 2613–2617.
[14] S. F. Steinbach, “Lexical Distances Between Europe’s Languages,” alternativetransport.wordpress.com, 2016. [Online]. Available: https://alternativetransport.wordpress.com/2015/05/05/34/ [Accessed: May 2025].
[15] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *Proc. 3rd International Conference on Learning Representations (ICLR)*, 2015.